

Углубленное машинное обучение и искусственный интеллект в Python

Патласов Дмитрий Александрович, ПГНИУ
dmitriypatlasov@gmail.com

Что такое NLP?

NLP - область лингвистики и машинного обучения, которая изучает все, что связано с естественными языками. Главная цель NLP не просто понимать отдельные слова, но и иметь возможность понимать контекст, в котором эти слова находятся.

Список типичных NLP-задач с некоторыми примерами:

- Классификация предложений: определить эмоциональную окраску отзыва, детектировать среди входящих писем спам, определить грамматическую правильность предложения или даже проверить, являются ли два предложения связанными между собой логически
- Классификация каждого слова в предложении: вычленив грамматические составляющие предложения (существительное, глагол, прилагательное) или определить именованные сущности (персона, локация, организация)
- Генерация текста: закончить предложение на основе некоторого запроса, заполнить пропуски в тексте, содержащем замаскированные слова
- Сформулировать ответ на вопрос: получить ответ на заданный по тексту вопрос
- Сгенерировать новое предложение исходя из предложенного: перевести текст с одного языка на другой, выполнить автоматическое реферирование текста

NLP не ограничивается только письменным текстом. Есть множество сложных задач, связанных с распознаванием речи и компьютерным зрением, таких как транскрибирование аудио или описание изображений.

Почему это сложно?

Компьютеры не обрабатывают информацию так же, как люди. Например, когда мы читаем предложение «Я голоден», мы можем легко понять его значение. Точно так же, имея два предложения, такие как «Я голоден» и «Мне грустно», мы можем легко определить, насколько они похожи. Для моделей машинного обучения (ML) такие задачи сложнее. Текст должен быть обработан так, чтобы модель могла учиться на нем. А поскольку язык сложен, нам нужно тщательно продумать, как должна выполняться эта обработка. Было проведено много исследований того, как представлять текст, и мы рассмотрим некоторые методы в следующей главе.

Что такое LLM и как она "думает"

Прежде чем строить приложения на LLM, нужно понять, что именно вы вызываете. Без этого вы будете удивляться "галлюцинациям", непредсказуемым ответам и считать за API.

Главное

LLM — это вероятностная модель следующего токена. Не базы знаний, не оракул, не интеллект. Модель смотрит на последовательность токенов и предсказывает распределение вероятностей для следующего. Далее из этого распределения сэмпляется один токен — и так до конца.

Из этого следуют все её свойства: она не "знает" фактов в смысле БД, она не умеет считать без подсказок, она склонна продолжать паттерн, который видит во вводе.

Токенизация

LLM работает не со словами и не с символами, а с токенами. Слово "машинное обучение" в GPT-4 — это примерно 4–5 токенов, в русском тексте — больше, чем в английском. Это напрямую влияет на стоимость и качество.

```
import tiktoken
enc = tiktoken.encoding_for_model("gpt-4o")
print(enc.encode("машинное обучение"))
# [122029, 30143, 11562, 11424, 27634]
print(len(enc.encode("machine learning")))
# 2
```



Следствие: русские промпты дороже английских в 2–4 раза при той же информации.

Контекстное окно

Это максимум токенов, которые модель может обработать за один вызов (промпт + ответ). У современных моделей — от 128k до 2M токенов. Но "вмещается" ≠ "использует эффективно". Lost-in-the-middle: модель хуже использует информацию из середины длинного контекста.

Sampling: температура, top_p, top_k

- **temperature=0** — детерминированный (почти) вывод, всегда самый вероятный токен. Для классификации, извлечения данных.
- **temperature=0.7–1.0** — креативность, разнообразие. Для генерации текста, brainstorming.
- **top_p=0.9** — отсекает маловероятные токены. Используйте либо temperature, либо top_p, не оба сразу.

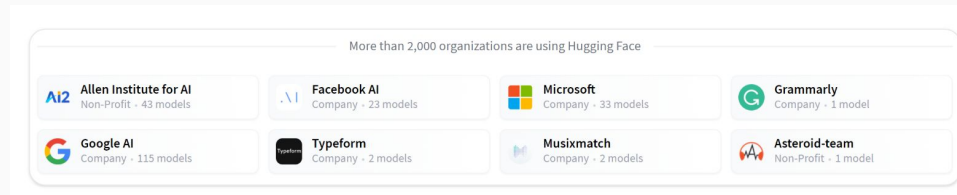
Антипаттерны

- Считать LLM источником истины. Она генерирует правдоподобный текст, а не правду.
- Просить LLM "посчитать" большие числа без code interpreter / tools. Она не калькулятор.
- Полагаться на temperature=0 как на гарантию воспроизводимости. На уровне API всё равно есть недетерминизм.
- Игнорировать токенизацию при работе с не-английскими языками.

Трансформеры повсюду!

Библиотека 🙌 Transformers предоставляет различную функциональность для создания и использования этих моделей. Model Hub содержит тысячи предобученных моделей, которые может скачать и использовать любой.

Вы также можете загружать свои модели на Model Hub!



Самый базовый объект библиотеки  Transformers - pipeline(). Он соединяет в себе необходимые шаги по предобработке и постобработке данных и позволяет передавать модели на вход текст, а на выходе получать читаемый ответ:

Самый базовый объект библиотеки  Transformers - pipeline(). Он соединяет в себе необходимые шаги по предобработке и постобработке данных и позволяет передавать модели на вход текст, а на выходе получать читаемый ответ:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
classifier("I've been waiting for a HuggingFace course my whole life.")
```

```
[{'label': 'POSITIVE', 'score': 0.9598047137260437}]
```

Мы можем передать на вход сразу несколько предложений!

```
classifier(
    ["I've been waiting for a HuggingFace course my whole life.", "I hate this so much!"]
)
```

```
[{'label': 'POSITIVE', 'score': 0.9598047137260437},
 {'label': 'NEGATIVE', 'score': 0.9994558095932007}]
```

По умолчанию этот пайплайн выбирает специальную модель, которая была предобучена для оценки тональности предложений на английском языке. Модель загружается и кэшируется когда вы создаете объект classifier. Если вы перезапустите команду, будет использована кэшированная модель, т.е. загрузки новой модели не произойдет.

В процессе обработки пайплайном текста, который вы ему передали, есть три главных шага:

1. Текст предобрабатывается в формат, который понятен модели.
2. Предобработанный текст передается в модель.
3. Результаты модели проходят через постобработку и вы получаете читаемый ответ.

Pipelines

Parameters

- **task** (`str`) — The task defining which pipeline will be returned. Currently accepted tasks are:
 - `"audio-classification"`: will return a [AudioClassificationPipeline](#).
 - `"automatic-speech-recognition"`: will return a [AutomaticSpeechRecognitionPipeline](#).
 - `"depth-estimation"`: will return a [DepthEstimationPipeline](#).
 - `"document-question-answering"`: will return a [DocumentQuestionAnsweringPipeline](#).
 - `"feature-extraction"`: will return a [FeatureExtractionPipeline](#).
 - `"fill-mask"`: will return a [FillMaskPipeline](#).
 - `"image-classification"`: will return a [ImageClassificationPipeline](#).
 - `"image-feature-extraction"`: will return an [ImageFeatureExtractionPipeline](#).
 - `"image-segmentation"`: will return a [ImageSegmentationPipeline](#).
 - `"image-text-to-text"`: will return a [ImageTextToTextPipeline](#).
 - `"keypoint-matching"`: will return a [KeypointMatchingPipeline](#).
 - `"mask-generation"`: will return a [MaskGenerationPipeline](#).
 - `"object-detection"`: will return a [ObjectDetectionPipeline](#).
 - `"table-question-answering"`: will return a [TableQuestionAnsweringPipeline](#).
 - `"text-classification"` (alias `"sentiment-analysis"` available): will return a [TextClassificationPipeline](#).
 - `"text-generation"`: will return a [TextGenerationPipeline](#).
 - `"text-to-audio"` (alias `"text-to-speech"` available): will return a [TextToAudioPipeline](#).
 - `"token-classification"` (alias `"ner"` available): will return a [TokenClassificationPipeline](#).
 - `"video-classification"`: will return a [VideoClassificationPipeline](#).
 - `"zero-shot-classification"`: will return a [ZeroShotClassificationPipeline](#).
 - `"zero-shot-image-classification"`: will return a [ZeroShotImageClassificationPipeline](#).
 - `"zero-shot-audio-classification"`: will return a [ZeroShotAudioClassificationPipeline](#).
 - `"zero-shot-object-detection"`: will return a [ZeroShotObjectDetectionPipeline](#).
- **model** (`str` or [PreTrainedModel](#), *optional*) — The model that will be used by the pipeline to make predictions. This can be a model identifier or an actual instance of a pretrained model inheriting from [PreTrainedModel](#).

Zero-shot classification

Мы начнем с более сложной задачи, где нам нужно классифицировать тексты, которые не были размечены (для них нет ответов - позитивный или негативный, агрессивный или нейтральный и тд). Это распространенный сценарий в реальных проектах, потому что разметка текста обычно занимает много времени и требует знаний в предметной области. Для этого варианта использования очень мощным является пайплайн `zero-shot-classification`: он позволяет указать, какие метки использовать для классификации, поэтому вам не нужно полагаться на метки предварительно обученной модели. Вы уже видели, как модель может классифицировать предложение как позитивное или негативное, используя эти две метки, но она также может классифицировать текст, используя любой другой набор меток, который вам подходит.

```
from transformers import pipeline

classifier = pipeline("zero-shot-classification")
classifier(
    "This is a course about the Transformers library",
    candidate_labels=["education", "politics", "business"],
)

{'sequence': 'This is a course about the Transformers library',
 'labels': ['education', 'business', 'politics'],
 'scores': [0.8445963859558105, 0.111976258456707, 0.043427448719739914]}
```

Этот пайплайн называется *zero-shot* потому, что вам нет необходимости дообучать модель для использования. Она может вернуть вам оценки вероятности для любого списка меток, который вы хотите!

Генерация текста

Теперь давайте взглянем на пайплайн генерации текста (англ. text generation). Главная идея заключается в следующем: вы передаете на вход модели небольшой фрагмент текста, а модель будет продолжать его. Это похоже на предсказание следующего слова в клавиатурах различных смартфонов. Генерация текста содержит в себе элемент случайности, поэтому ваш результат может отличаться от того, который приведен ниже в примере.

```
from transformers import pipeline

generator = pipeline("text-generation")
generator("In this course, we will teach you how to")
```

```
[{'generated_text': 'In this course, we will teach you how to understand and use '
                    'data flow and data interchange when handling user data. We '
                    'will be working with one or more of the most commonly used '
                    'data flows – data flows of various types, as seen by the '
                    'HTTP'}]
```

Вы можете указать, сколько разных «ответов» сгенерирует модель, задав параметр `num_return_sequences`. Чтобы изменить длину ответной последовательности, нужно передать значение в аргумент `max_length`.

Использование произвольной модели из Hub в пайплайне

Предыдущие примеры использовали модель по умолчанию для решения конкретной задачи, но у вас есть возможность выбрать произвольную модель из Hub и передать ее в пайплайн для конкретной задачи. Например, для генерации текста. Перейдите по ссылке [Model Hub](#) и кликните на соответствующий тег слева, чтобы получить список доступных для этой задачи моделей. Вы должны увидеть страницу, подобную [этой](#).

Давайте попробуем модель `distilgpt2`! Тут показано, как загрузить ее в такой же пайплайн, как в примерах выше:

```
from transformers import pipeline

generator = pipeline("text-generation", model="distilgpt2")
generator(
    "In this course, we will teach you how to",
    max_length=30,
    num_return_sequences=2,
)
```

```
[{'generated_text': 'In this course, we will teach you how to manipulate the world and '
                    'move your mental and physical capabilities to your advantage.'},
 {'generated_text': 'In this course, we will teach you how to become an expert and '
                    'practice realtime, and with a hands on experience on both real '
                    'time and real'}]
```

Вы можете уточнить, для какого языка вам нужна модель, щелкнув на языковые теги, и выбрать ту, которая будет генерировать текст на другом языке. На Model Hub даже есть предобученные модели для многоязычных задач.

Как только вы выберете модель, вы увидите, что есть виджет, позволяющий вам попробовать ее прямо на сайте. Таким образом, вы можете быстро протестировать возможности модели перед ее загрузкой.

Заполнение пропусков

Следующая задача, на которую мы обратим внимание, связана с заполнением пропусков в тексте (англ. mask filling). Идея очень проста, мы продемонстрируем ее на простом тексте:

```
from transformers import pipeline

unmasker = pipeline("fill-mask")
unmasker("This course will teach you all about <mask> models.", top_k=2)
```

```
[{'sequence': 'This course will teach you all about mathematical models.',
  'score': 0.19619831442832947,
  'token': 30412,
  'token_str': ' mathematical'},
 {'sequence': 'This course will teach you all about computational models.',
  'score': 0.04052725434303284,
  'token': 38163,
  'token_str': ' computational'}]
```

Аргумент `top_k` указывает, сколько вариантов для пропущенного слова будет отображено. Обратите внимание, что модель заполнит пропуск на месте слова `<mask>`, которое часто интерпретируют как *mask token (токен-маска)*.

Другие модели могут использовать другие токены для обозначения пропуска, всегда лучше проверять это. Один из способов сделать это - обратиться к виджету для соответствующей модели.

Распознавание именованных сущностей (NER)

Распознавание именованных сущностей (англ. named entity recognition) - это задача, в которой модели необходимо найти части текста, соответствующие некоторым сущностям, например: персонам, местам, организациям. Давайте посмотрим на пример:

```
from transformers import pipeline

ner = pipeline("ner", grouped_entities=True)
ner("My name is Sylvain and I work at Hugging Face in Brooklyn.")
```

```
[{'entity_group': 'PER', 'score': 0.99816, 'word': 'Sylvain', 'start': 11, 'end': 18},
 {'entity_group': 'ORG', 'score': 0.97960, 'word': 'Hugging Face', 'start': 33, 'end': 45},
 {'entity_group': 'LOC', 'score': 0.99321, 'word': 'Brooklyn', 'start': 49, 'end': 57}]
```

В этом примере модель корректно обозначила Sylvain как персону (PER), Hugging Face как организацию (ORG) и Brooklyn как локацию (LOC).

Мы передали в пайплайн аргумент `grouped_entities=True` для того, чтобы модель сгруппировала части предложения, соответствующие одной сущности: в данном случае модель объединила “Hugging” и “Face” несмотря на то, что название организации состоит из двух слов. На самом деле, как мы увидим в следующей главе, препроцессинг делит даже отдельные слова на несколько частей. Например, Sylvain будет разделено на 4 части: S, ##y1, ##va, and ##in. На этапе постпроцессинга пайплайн успешно объединит эти части.

Вопросно- ответные системы

Пайплайн question-answering позволяет сгенерировать ответ на вопрос по данному контексту:

```
from transformers import pipeline

question_answerer = pipeline("question-answering")
question_answerer(
    question="Where do I work?",
    context="My name is Sylvain and I work at Hugging Face in Brooklyn",
)

{'score': 0.6385916471481323, 'start': 33, 'end': 45, 'answer': 'Hugging Face'}
```

Обратите внимание, что пайплайн извлекает информацию для ответа из переданного ему контекста

Автоматическое реферирование (саммаризация)

Автоматическое реферирование (англ. summarization) - задача, в которой необходимо сократить объем текста, но при этом сохранить все важные аспекты (или большинство из них) изначального текста. Вот пример:

```
from transformers import pipeline

summarizer = pipeline("summarization")

summarizer(
    """
    America has changed dramatically during recent years. Not only has the number of
    graduates in traditional engineering disciplines such as mechanical, civil,
    electrical, chemical, and aeronautical engineering declined, but in most of
    the premier American universities engineering curricula now concentrate on
    and encourage largely the study of engineering science. As a result, there
    are declining offerings in engineering subjects dealing with infrastructure,
    the environment, and related issues, and greater concentration on high
    technology subjects, largely supporting increasingly complex scientific
    developments. While the latter is important, it should not be at the expense
    of more traditional engineering.

    Rapidly developing economies such as China and India, as well as other
    industrial countries in Europe and Asia, continue to encourage and advance
    the teaching of engineering. Both China and India, respectively, graduate
    six and eight times as many traditional engineers as does the United States.
    Other industrial countries at minimum maintain their output, while America
    suffers an increasingly serious decline in the number of engineering graduates
    and a lack of well-educated engineers.
    """)
```

```
[{'summary_text': ' America has changed dramatically during recent years . The '
                  'number of engineering graduates in the U.S. has declined in '
                  'traditional engineering disciplines such as mechanical, civil '
                  ', electrical, chemical, and aeronautical engineering . Rapidly '
                  'developing economies such as China and India, as well as other '
                  'industrial countries in Europe and Asia, continue to encourage '
                  'and advance engineering .'}]
```

Так же, как и в задаче генерации текста, вы можете указать максимальную длину `max_length` или минимальную длину `min_length` результата.

Для перевода вы можете использовать модель по умолчанию просто указав пару языков, между которыми будет осуществляться перевод (например, "translation_en_to_fr"). Однако самый простой способ - попробовать использовать Model Hub. Здесь мы попробуем перевести текст с французского на английский язык:

```
from transformers import pipeline

translator = pipeline("translation", model="Helsinki-NLP/opus-mt-fr-en")
translator("Ce cours est produit par Hugging Face.")
```

```
[{'translation_text': 'This course is produced by Hugging Face.'}]
```

Так же, как и в задачах генерации и автоматического реферирования текста, вы можете указать максимальную длину `max_length` или минимальную длину `min_length` результата.

Fine-tuning: LoRA, QLoRA, когда это нужно

Fine-tuning — это адаптация предобученной модели под конкретную задачу или стиль. Звучит соблазнительно, но в 80% случаев лучше начать с промпта + RAG. Этот урок про оставшиеся 20%, где fine-tuning реально нужен.

Когда fine-tuning оправдан

- Стабильный формат вывода, который сложно описать промптом.
- Специфический стиль, тон, "голос бренда".
- Сужение задачи: модель должна делать только X, никаких длинных рассуждений.
- Очень большие промпты, которые хочется "впечь" в модель.
- Локальная маленькая модель должна выйти на качество большой облачной на узкой задаче.

Когда fine-tuning НЕ нужен

- Нужно добавить знания. Это RAG, не fine-tuning.
- Свежие данные. Тюненая модель устаревает быстро.
- Промпт ещё не оптимизирован. Сначала промпт + few-shot.
- Меньше 200–500 качественных примеров. Будет переобучение.

Fine-tuning: LoRA, QLoRA, когда это нужно

LoRA и QLoRA

Полный fine-tuning большой модели требует огромных ресурсов. LoRA (Low-Rank Adaptation) обучает маленькие "адаптеры" поверх замороженных весов — нужны единицы ГБ VRAM. QLoRA добавляет квантование базовой модели до 4 бит — позволяет тюнить 7B–13B на одной потребительской GPU.

```
from peft import LoraConfig, get_peft_model
lora_config = LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj", "v_proj"])
model = get_peft_model(base_model, lora_config)
# обучаем только адаптеры
```



Подготовка данных

Это 80% работы. Качество датасета важнее размера.

- Формат: пары (вход, ожидаемый выход) или (системный промпт, вход, выход).
- Размер: минимум 200, лучше 1000–5000 примеров.
- Качество: каждый пример вычитан, edge cases представлены, нет противоречий.
- Разделение: train / val / test (70/15/15).

```
{"messages": [{"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]}
```



Гиперпараметры

Стандартные стартовые значения:

- LR: $1e-4$ для LoRA, $1e-5$ для полного fine-tuning.
- Epochs: 1–3, больше — переобучение.
- LoRA rank: 8–32. Больше — мощнее, но рискованнее.
- Batch size: что влезет, через gradient accumulation.

Логируйте train loss, val loss, метрики на тестовом наборе.

Оценка

Тюненная модель — это новая модель. Ей нужны те же evals, что и для prompt-based решения. Сравните с baseline (промпт + большая модель) по: качеству, стоимости, latency.

Часто fine-tuned маленькая модель догоняет большую по качеству на узкой задаче, выигрывая в 5–10× по стоимости и latency.

Облачный fine-tuning

OpenAI, Anthropic, Google предлагают fine-tuning через API. Удобно: не нужно своё железо. Минусы: vendor lock-in, ваши данные у провайдера, дороже.

Антипаттерны

- "Натюним модель на наших документах, чтобы она их знала". Не работает. Это RAG.
- Маленький грязный датасет. Получите шум.
- Не сравнивать с baseline. "Стало лучше" — не метрика без замеров.
- Тюнить только на happy path. На edge cases модель развалится.
- Хранить тюненую модель без версии и без датасета, на котором её обучили.

Зачем fine-tuning

Обучать большую модель с нуля могут позволить себе 5 компаний в мире. Все остальные используют **transfer learning** — берут чужую модель и адаптируют её. Это:

- В 10-1000 раз дешевле.
- Требует на 2-3 порядка меньше данных.
- Часто даёт результат лучше, чем модель с нуля на этих же данных.

Виды transfer learning

1. Feature extraction. Замораживаем всю предобученную модель, используем её как «извлекатель признаков», обучаем только новую голову.

```
import torch.nn as nn
from torchvision import models

resnet = models.resnet50(pretrained=True)
for p in resnet.parameters():
    p.requires_grad = False
resnet.fc = nn.Linear(2048, 10) # 10 классов своих
# обучаем только resnet.fc.parameters()
```



2. Full fine-tuning. Размораживаем всю модель, обучаем все параметры с **маленьким** learning rate (`1e-5` для трансформеров). Лучшее качество, но дорогое.

3. Discriminative fine-tuning. Разные lr для разных слоёв: меньший — для нижних (общие фичи), больший — для верхних (специфика задачи).

4. PEFT (Parameter-Efficient Fine-Tuning). Обновляем не все веса, а маленький набор дополнительных параметров. Главный представитель — LoRA.

LoRA: математика и интуиция

LoRA (Low-Rank Adaptation, 2021) заметил, что обновления весов при fine-tuning часто имеют **низкий ранг**. Значит, можно представить обновление как произведение двух маленьких матриц:

$$W_{\text{new}} = W_{\text{frozen}} + \Delta W, \quad \Delta W = B \cdot A$$

где A — матрица $r \times k$, B — матрица $d \times r$, и $r \ll \min(d, k)$ (обычно $r = 4-64$).

Параметров вместо $d \cdot k$ становится $r(d+k)$ — **в десятки раз меньше**. W_{frozen} не обучается, обучаются только A и B .

Что это даёт:

- Можно fine-tune 7B модель на одной 24GB GPU.
- LoRA-веса весят 5-200 MB вместо 14 GB — удобно хранить много адаптеров под разные задачи.
- На inference можно «склеить» $W + BA$ обратно — нулевые накладные расходы.

```
# Псевдокод LoRA-обёртки
class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, r=8, alpha=16):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False
        self.A = nn.Parameter(torch.randn(r, base.in_features) * 0.01)
        self.B = nn.Parameter(torch.zeros(base.out_features, r))
        self.scale = alpha / r

    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale
```



Заметьте инициализацию: B нулями, A маленькими случайными. В начале обучения $BA = 0 \rightarrow$ модель идентична оригинальной. Это критично для стабильности.

Готовые инструменты: HuggingFace + PEFT

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model

model = AutoModelForCausalLM.from_pretrained("mistralai/Mistral-7B-v0.1", load_in_4bit=True)
tok = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-v0.1")

config = LoraConfig(
    r=16, lora_alpha=32,
    target_modules=["q_proj", "v_proj"], # обычно достаточно
    lora_dropout=0.05, bias="none",
)
model = get_peft_model(model, config)
model.print_trainable_parameters()
# trainable: ~10M из ~7000M — 0.15%
```



С квантизацией в 4 бита (`load_in_4bit=True`) это QLoRA — можно fine-tune Mistral 7B даже на бесплатной T4 в Colab.

Когда какой подход

Ситуация	Что использовать
100-1000 примеров своих данных	Feature extraction или LoRA
1000-10 000 примеров	LoRA
10 000-100 000 примеров	LoRA с большим <code>r</code> или full fine-tune
100 000+ примеров и большой бюджет	Full fine-tune
Нужно много задач на одной модели	LoRA адаптеры (по адаптеру на задачу)

SFT и instruction-tuning

Современные LLM проходят 3 стадии:

1. **Pretraining** — на огромном корпусе текста. Это «знания».
2. **SFT (Supervised Fine-Tuning)** — на парах «инструкция → ответ». Это «умение следовать инструкциям».
3. **RLHF / DPO** — выравнивание под человеческие предпочтения.

Большинство практических задач решается на этапе 2: возьмите Mistral/Llama, соберите 1-10k примеров под вашу задачу (например, «вопрос по нашему API → структурированный ответ»), запустите LoRA-SFT. Часто этого достаточно, чтобы обогнать GPT-4 на узкой задаче.

Главные грабли

- **Catastrophic forgetting.** При full fine-tuning модель может забыть общие знания. Решения: меньший lr, LoRA, замораживание нижних слоёв.
- **Утечка данных.** Если test-данные похожи на train — метрики будут наврут. Делите по времени или по пользователям, не случайно.
- **Слишком много эпох.** Для fine-tuning большой модели обычно достаточно 1-3 эпох. Дальше — переобучение.
- **Неправильный формат данных.** Для instruction-tuning важно точно соблюдать chat template модели. Mistral, Llama-3, Qwen имеют разные форматы.

Что значит «положить модель в прод»

Это минимум 5 проблем, не одна:

1. **Сериализация.** Сохранить веса так, чтобы их можно было загрузить без исходного кода тренировки.
2. **Скорость инференса.** `model(x)` в обучении и в проде — это разные требования.
3. **Память.** 7B-модель в fp32 — это 28 GB. На прод-сервере столько может не быть.
4. **API.** Кто-то должен вызвать модель по HTTP и получить результат.
5. **Мониторинг.** Понимать, когда модель «поплыла» и пора переобучать.

Шаг 1. Сохранение и загрузка

```
# Веса — лёгкий, безопасный способ
torch.save(model.state_dict(), 'model.pt')

# Загрузка
model = MyModel()
model.load_state_dict(torch.load('model.pt', map_location='cpu'))
model.eval()
```



Не используйте `torch.save(model)` (без `state_dict`). Он сохраняет Python-объект, ломается при изменении кода и небезопасен при загрузке чужих файлов (могут содержать произвольный код).

Шаг 2. Переход на ONNX

ONNX (Open Neural Network Exchange) — формат, который понимают почти все runtime'ы: ONNX Runtime, TensorRT, OpenVINO, CoreML. Один `.onnx` файл — и модель крутится на CPU, GPU, мобильном устройстве, в браузере.

```
dummy = torch.randn(1, 3, 224, 224)
torch.onnx.export(
    model, dummy, "model.onnx",
    input_names=["input"], output_names=["output"],
    dynamic_axes={"input": {0: "batch"}, "output": {0: "batch"}},
    opset_version=17,
)

# Инференс через ONNX Runtime
import onnxruntime as ort
sess = ort.InferenceSession("model.onnx", providers=["CPUExecutionProvider"])
out = sess.run(None, {"input": dummy.numpy()})
```

На многих моделях ONNX Runtime даёт 2-5× ускорение на CPU по сравнению с обычным PyTorch.



Шаг 3. Квантизация

Идея — хранить веса не в `float32`, а в `int8` или `int4`. Размер модели падает в 4-8 раз, скорость на CPU/edge — растёт.

Post-training dynamic quantization (простейший вариант):

```
import torch.quantization as tq
quantized = tq.quantize_dynamic(model, {nn.Linear, nn.LSTM}, dtype=torch.qint8)
```



Для LLM используют более продвинутые методы: `bitsandbytes` (8 и 4 бита), `GPTQ` (4 бита, GPU), `AWQ` (Activation-aware), `GGUF` (формат llama.cpp для CPU).

Типичная просадка качества при `int8` квантизации — 0-1%. При `int4` — 1-3%. Это часто **приемлемая цена** за 4× ускорение и снижение памяти.

Шаг 4. Простой API на FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel
import onnxruntime as ort, numpy as np

app = FastAPI()
sess = ort.InferenceSession("model.onnx", providers=["CPUExecutionProvider"])

class Input(BaseModel):
    features: list[float]

@app.post("/predict")
def predict(inp: Input):
    x = np.array([inp.features], dtype=np.float32)
    y = sess.run(None, {"input": x})[0]
    return {"prediction": y[0].tolist()}
```

Запуск: `uvicorn app:app --host 0.0.0.0 --port 8000`. Контейнеризация — Dockerfile из 5 строк. Готово.

Шаг 5. Batch inference и динамический батчинг

В проде запросы приходят по одному, но GPU работает эффективно только на батчах. Решение — **сервер с динамическим батчингом** (NVIDIA Triton, vLLM, TGI). Сервер сам собирает запросы, накопившиеся за 10-50 мс, в один батч.

Без батчинга 1 запрос = 100% занятость GPU на коротком отрезке. С батчингом — 16 запросов одновременно за то же время.

Шаг 6. KV-кеш для LLM

При автогрессивной генерации каждый новый токен требует пересчёта attention по всему контексту. KV-кеш хранит уже посчитанные `k` и `v` от предыдущих токенов — пересчитывать нужно только последний.

Без кеша генерация N токенов = $O(N^2)$ операций. С кешем — $O(N)$. На длинных контекстах разница в 100× и больше. В `transformers` это `use_cache=True` (по умолчанию включено).

Шаг 7. Inference-движки для LLM

Не пишите свой LLM-сервер. Используйте готовое:

- **vLLM** — высокая пропускная способность, PagedAttention, continuous batching.
- **TGI (Text Generation Inference)** — HuggingFace, продакшн-ready.
- **llama.cpp / ollama** — для CPU и edge, формат GGUF.
- **Triton Inference Server** — универсальный, для любых моделей.
- **TensorRT-LLM** — для NVIDIA GPU, максимальная скорость.

Выбор: для CPU/локально — llama.cpp; для одного сервера с GPU — vLLM; для большой нагрузки в облаке — Triton или TGI.

Узнайте, какие локальные модели потянет ваш ПК

<https://whatmodelscanirun.com/>

Как работает:

- указываете GPU, VRAM и RAM
- получаете список моделей, которые нормально запустятся на вашем ПК
- видите квантование, примерную скорость и контекстное окно
- поддерживается железо NVIDIA, AMD, Intel и Apple

Особенно удобно, если собираете ИИ-агентов, тестируете локальные модели или выбираете железо под inference.

Больше не нужно вручную считать память и перебирать модели наугад.



What Models?

Pick the right model for your GPU — in seconds.

SELECT YOUR GPU

GeForce RTX 5090 (32 GB) ×

GPUS

1

VRAM (GB)

or e.g. 8

MINIMUM CONTEXT WINDOW

128K ▾

MINIMUM TOKENS/SEC

Any ▾

REQUIRED FEATURES

Any ▾

SYSTEM RAM (optional) ⓘ

e.g. 3 GB

Using **32 GB** VRAM, need at least **128K** context

Results

90 of 138 model configurations can run on **32 GB** with at least **128K** context — sorted by quality

Ranked by **MMLU** ⓘ

Copy link

RUNS WELL

Qwen3.6 27B 27B

Excellent Vision Tool use Reasoning

MMLU 86.2

Q4_K_M

CONTEXT 242K SPEED ~102 tok/s

Qwen3.6 35B A3B 35B

Excellent Vision Tool use Reasoning

MMLU 85.2

Q4_K_M

CONTEXT SPEED

Калькулятор расчета инференса и дообучения LLM

Бесплатно

(<https://apxml.com/tools/vram-calculator>),
помогает понять, какой GPU нужен под
конкретную задачу до того как потрачен
бюджет.

Выбираете параметры инференса:
архитектуру модели, тип квантования,
sequence length и batch size.

Параметры расчета дообучения
учитывают: конфиг датасета (количество
сэмпл, среднее токенов на сэмпл,
эпохи) и использование оптимизаторов.
Плюс, еще посчитает время обучения.

Цифры часто получаются чуть выше
реального потребления (что даже
хорошо), но точности до гигабайта ждать
не стоит.

LLM Inference: VRAM & Performance Calculator

[Share This Config](#)

[Submit Benchmark](#)

Inference

Fine-tuning

Select Model *

Qwen2-7B

Inference Quantization

Precision for model weights during inference. Lower uses less VRAM but may affect quality.

Q4KM

KV Cache Quantization

KV Cache precision. Lower values reduce VRAM, especially for long sequences.

FP16 / BF16 (Default)

Hardware Configuration

Select your GPU or set custom VRAM

RTX 5070 Ti (16GB)

Num GPUs

Devices for parallel inference

1

Input Parameters

Slider

Batch Size: ☐ Log Scale

1

Inputs processed simultaneously per step (affects throughput & latency)

1

8

16

32

Sequence Length: 1024

Max tokens per input; impacts KV cache (also affected by attention structure) & activations.

1

8K

16K

33K

66K

131K

Concurrent Users: ☐ Log Scale

1

Number of users running inference simultaneously (affects memory usage and per-user performance)

1

4

8

16

32

Performance & Memory Results

33.2%

VRAM

OKAY

5,31 GB

of 16 GB VRAM

① Generation Speed: ~118 tok/sec ①

(~8.5 ms/token latency)

① Time to First Token: ~61ms ①

⚡ Total Throughput: ~118 tok/sec ①

Profile: Optimized for Lowest Latency ①

Alibaba Qwen2-7B

Weights: **Q4 K M**

KV Cache: **FP16**

Attention Structure: **GQA** ①

Positional Embeddings: **ROPE**

Mode: Inference | Batch: 1

Memory Allocation

QLoRA Fine-tuning: VRAM Calculator

[Share This Config](#)

Inference

Fine-tuning

Training Mode

Choose how the model will be trained

QLoRA

Select Model *

Qwen2-7B

Base Model Precision

QLoRA uses 4-bit base model weights (fixed).

4-bit (Base)

LoRA Rank (r)

Dimension of the low-rank adapters (higher rank = more memory/potential quality)

16

Hardware Configuration

Select your GPU or set custom VRAM

RTX 5070 Ti (16GB)

Num GPUs

Devices for parallel training

1

Input Parameters

Slider

Batch Size: 1

Samples per optimization step



Sequence Length: 1024

Max tokens per training sample (affects activations memory)



Performance & Memory Results



OKAY

7,45 GB

of 16 GB VRAM

Training Speed: ~12 tok/sec

Samples/sec: ~0.01

Steps/sec: ~0.01

Total Tokens: 15.4M

Est. Training Time: 14.8 days

Alibaba Qwen2-7B

Method: QLoRA

Base Precision: 4-bit

LoRA Rank: R=16

Positional Embeddings: RoPE

Mode: Fine-tuning (QLoRA) | Batch: 1